

Programming Fundamentals

SHORT NOTES · CSEG1044 · UPES BTech CSE

Java Primary | Semester 4 | 50 Theory + 50 Practical

Topic	Marks / Notes
JDBC / IO Streams / GUI Swing	20 marks each — HIGHEST PRIORITY
Collections + Multithreading	10 marks each — CRITICAL
Inheritance + Exceptions	10 marks each — HIGH
Data Types + Strings	4 marks — MEDIUM
Study Time	18–22 hours total

1 · LANGUAGE BASICS

Language	Entry Point	Print	Comments
Java	public static void main(String[] args)	System.out.println()	// and /* */
C	int main()	printf()	// and /* */
Python	if __name__ == '__main__':	print()	# and ''' '''

2 · JAVA DATA TYPES (fixed size, platform-independent)

Type	Size	Default	Key Rule / Range
byte	1 B	0	-128 to 127
short	2 B	0	-32768 to 32767
int	4 B	0	~2.1 billion
long	8 B	0L	Suffix L required: 100L
float	4 B	0.0f	Suffix f required: 3.14f
double	8 B	0.0	Default decimal type
char	2 B	\u0000	Unicode (not ASCII!)
boolean	1 bit	false	true / false only

■ float f = 3.14; → **ERROR** — must be: float f = 3.14f;

■ long l = 100000000000; → **ERROR** — must be: long l = 100000000000L;

3 · OPERATORS & PRECEDENCE (4–6 marks, 5/5 years)

Priority	Operators	Note
1 (High)	() [] .	Grouping / member access
2	++ -- (postfix)	Use value FIRST, then change
3	++ -- ! (prefix)	Change FIRST, then use value
4	* / %	Integer division truncates in Java!
5	+ -	
6	< <= > >=	Relational
7	== !=	Equality
8	&&	Logical AND
9		Logical OR
10 (Low)	= += -= *=	Assignment (right-to-left)

Pre / Post Increment (4/5 years — 4 marks)

```
int x = 5, y;  
y = x++; // y=5, x=6 — use FIRST, then increment  
y = ++x; // x=7, y=7 — increment FIRST, then use
```

■ Java int division truncates: 10/3 = 3. Cast needed: (double)10/3 = 3.333

■ if (x = 5) is assignment — always true! Use: if (x == 5)

4 · CONTROL STRUCTURES (5/5 years)

```
// IF-ELSE
if (x>0){ } else if (x<0){ } else{ }

// TERNARY
int max = (a > b) ? a : b;

// SWITCH – always add break!
switch(day) {
    case 1: print("Mon"); break;
    case 2: print("Tue"); break;
    default: print("Other");
}
```

```
// FOR
for (int i=0; i<5; i++) { }

// WHILE
while (condition) { }

// DO-WHILE (runs at least ONCE)
do { } while (condition);

// FOR-EACH
for (int v : arr) { }
```

Break + Continue (5/5 years — output prediction)

```
for (int i=0; i<5; i++) {
    if (i==3) continue; // skip 3
    if (i==4) break;    // exit at 4
    System.out.print(i + " ");
}
// Output: 0 1 2
```

■ ■ 5 · ARRAYS & STRINGS (5/5 years)

Operation	Array	String
Declare	<code>int[] arr = {10,20,30};</code>	<code>String s = "hello";</code>
Length	<code>arr.length</code> (no parentheses!)	<code>s.length()</code> (with parentheses!)
Access	<code>arr[0] → 10</code>	<code>s.charAt(1) → 'e'</code>
Error	<code>arr[5]</code> on size-5 → Exception	<code>==</code> compares refs — use <code>.equals()</code>

Key String Methods

Method	Example	Output
<code>length()</code>	<code>"hello".length()</code>	5
<code>charAt(i)</code>	<code>"hello".charAt(1)</code>	'e'
<code>substring(a,b)</code>	<code>"hello".substring(1,3)</code>	"el"
<code>toUpperCase()</code>	<code>"hello".toUpperCase()</code>	"HELLO"
<code>equals(s)</code>	<code>"ab".equals("ab")</code>	true
<code>replace(a,b)</code>	<code>"hello".replace('l','r')</code>	"herro"
<code>split(regex)</code>	<code>"a,b,c".split(",")</code>	["a","b","c"]

String Immutability (5/5 years — 4 marks)

```
String s1 = "Advanced";
String s2 = s1 + "Programming"; // s2 = "AdvancedProgramming"
s1 = "Java" + "Language";       // s1 = "JavaLanguage"
s2.concat(s1);                  // DISCARDED – result not assigned!
System.out.println(s1);        // JavaLanguage
System.out.println(s2);        // AdvancedProgramming (UNCHANGED!)
```

■ ■ 6 · CLASSES & ENCAPSULATION (5/5 years — 8–10 marks)

```
public class Student {
    private int rollNo;           // private = encapsulation
    private String name;

    public Student(int r, String n) { // Constructor (same name, no return type)
        this.rollNo = r;           // 'this' disambiguates field vs param
        this.name = n;
    }

    public Student() { this(0, "Unknown"); } // Constructor chaining
    public String getName() { return name; } // Getter
    public void setName(String n) { this.name = n; } // Setter
    public void display() { System.out.println(rollNo + " " + name); }
}

// Usage:
Student s1 = new Student(101, "Alice");
s1.display();
```

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
(default)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Static vs Instance (3/5 years — output prediction)

```
class Counter {
    static int count = 0; // SHARED by all objects
    int id;
    Counter() { count++; id = count; }
}

Counter c1=new Counter(); Counter c2=new Counter(); Counter c3=new Counter();
System.out.println(Counter.count); // 3 (static NOT reset on new object!)
System.out.println(c1.id); // 1
```

■ ■ 7 · INHERITANCE & POLYMORPHISM (5/5 years — 10–20 marks)

Type	Structure	Java?	Keyword
Single	A → B	✓ Yes	extends
Multilevel	A → B → C	✓ Yes	extends
Hierarchical	A → B, A → C	✓ Yes	extends
Multiple	A + B → C	✗ No (classes)	implements (interfaces only)
Hybrid	Mix	✓ via Interfaces	implements

■ No multiple class inheritance → avoids Diamond Problem (ambiguous method resolution)

```
class Animal { String name; Animal(String n){name=n;} void eat(){}}
// Multilevel: Animal → Mammal → Dog
class Mammal extends Animal {
    Mammal(String n) { super(n); } // super() MUST be first line
}
class Dog extends Mammal {
    Dog(String n) { super(n); }
    void bark() { System.out.println(name + " barks"); }
}

// Constructor order: Dog() → super() Mammal() → super() Animal() runs first!
```

Overloading vs Overriding

Aspect	Overloading	Overriding
Location	Same class	Parent / child classes
Parameters	Must differ	Must be identical
Binding	Compile-time (static)	Runtime (dynamic)
Polymorphism	Compile-time	Runtime

@Override	Not used	Should annotate
-----------	----------	-----------------

```

// OVERRIDING + Runtime Polymorphism:
class Animal { void speak(){System.out.println("Some sound");} }
class Dog extends Animal {
    @Override void speak(){System.out.println("Woof!");}
}
Animal a = new Dog(); // Parent ref, child object
a.speak();             // Output: Woof! (runtime decides)

```

Abstract Class vs Interface

Aspect	Abstract Class	Interface
Methods	Abstract + Concrete	Abstract only (Java 7) / Default (Java 8+)
Variables	Any type	public static final (constants only)
Inheritance	Single (extends)	Multiple (implements A, B, C)
Constructor	Can have	No constructor
Use when	Shared code + partial impl.	Contract / multiple inheritance needed

```

abstract class Shape { abstract double area(); } // cannot instantiate
interface Drawable { void draw(); } // contract
interface Colorable { void setColor(String c); }

class Circle extends Shape implements Drawable, Colorable {
    double r; Circle(double r){this.r=r;}
    public double area() { return Math.PI*r*r; }
    public void draw() { System.out.println("Drawing circle"); }
    public void setColor(String c){ }
}

```

■ **final class = cannot subclass | final method = cannot override | final var = constant**

■ **final vs finally vs finalize: keyword | exception block | GC method (5/5 years — 4 marks)**

■ ■ 8 · EXCEPTION HANDLING (5/5 years — 4–10 marks)

Type	Examples	Detected	Must Handle?
Checked	IOException, FileNotFoundException	Compile-time	✓ Yes (try-catch or throws)
Unchecked	NullPointerException, ArithmeticException	Runtime	Optional
Error	StackOverflowError, OutOfMemoryError	Runtime	Usually unrecoverable

```

try {
    int[] arr = new int[5];
    arr[10] = 1; // throws ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array error: " + e.getMessage());
} catch (Exception e) { // catches any other exception
    System.out.println("General: " + e.getMessage());
} finally {
    System.out.println("ALWAYS runs – use for cleanup/close");
}

// throw – inside method body : throw new IllegalArgumentException("msg");
// throws – in method signature : void readFile() throws IOException { }

```

■ ■ 9 · MULTITHREADING (5/5 years — 4–10 marks)

State	How to Enter
New	new Thread()
Runnable	start()
Running	JVM scheduler picks it
Blocked	I/O or synchronized block
Waiting	wait(), join()
Terminated	run() completes or exception

```

// Way 1: extend Thread
class MyThread extends Thread {
    public void run() { for(int i=1;i<=5;i++) System.out.println(i); }
}

```

```
new MyThread().start();  
  
// Way 2: implement Runnable (PREFERRED – allows other extends)  
class MyRunnable implements Runnable {  
    public void run() { for(int i=1;i<=5;i++) System.out.println(i); }  
}  
  
new Thread(new MyRunnable()).start();  
  
// TRAP: calling start() twice → IllegalThreadStateException!  
// run() alone does NOT create a new thread!
```

```
// Synchronization – prevents data corruption on shared resource  
class Counter {  
    private int count = 0;  
    public synchronized void increment() { count++; } // one thread at a time  
}
```

■ ■ 10 · IO STREAMS & FILE HANDLING (4/5 years — 20 marks)

Stream	Classes	Use For
Byte Streams	FileInputStream / FileOutputStream	Binary data (images, audio)
Character Streams	FileReader / FileWriter	Text files
Buffered	BufferedReader / BufferedWriter	Efficient line-by-line reading

FileWriter + FileReader — Classic 20-mark Template

```
import java.io.*;

public class MovieFileManager {

    // WRITE
    static void writeMovies(String f) throws IOException {
        BufferedWriter bw = new BufferedWriter(new FileWriter(f));
        bw.write("Inception,2010,Nolan");
        bw.newLine();           // platform-independent newline
        bw.write("Tenet,2020,Nolan");
        bw.close();              // MUST close – flushes buffer to disk!
    }

    // APPEND (true = append mode)
    static void appendMovie(String f, String line) throws IOException {
        BufferedWriter bw = new BufferedWriter(new FileWriter(f, true));
        bw.write(line); bw.newLine(); bw.close();
    }

    // READ
    static void readMovies(String f) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader(f));
        String line;
        while ((line = br.readLine()) != null) // null = end of file
            System.out.println(line);
        br.close();
    }
}
```

■ Always close streams! Or use try-with-resources: try (BufferedWriter bw = new BufferedWriter(...)) { }

■ ■ 11 · COLLECTIONS FRAMEWORK (5/5 years — 10 marks)

Interface	Class	Order?	Duplicates?	Sorted?
List	ArrayList / LinkedList	Insertion	✓ Yes	✗ No
Set	HashSet	None	✗ No	✗ No
Set	LinkedHashSet	Insertion	✗ No	✗ No
SortedSet	TreeSet	Sorted	✗ No	✓ Yes
Map	HashMap	None	Keys:No	✗ No
SortedMap	TreeMap	By key	Keys:No	✓ Yes

ArrayList CRUD (5/5 years — 10 marks)

```
ArrayList list = new ArrayList<>();
list.add("Alice");           // CREATE
list.add("Bob");
list.get(0);                 // READ → "Alice"
list.set(1, "Charlie");      // UPDATE (index, value)
list.remove(0);              // DELETE by index
list.size();                 // 1 (after remove)
for (String s : list) System.out.println(s); // iterate
```

TreeSet Output Prediction (4/5 years — 4 marks)

```
TreeSet al = new TreeSet<>();
al.add("Programming"); al.add("Object"); al.add("Oriented");
al.add("Object"); // DUPLICATE – ignored!
for (String s : al) System.out.println(s);
// Output (SORTED alphabetically, NO duplicates):
// Object
// Oriented
// Programming
```

HashMap Key Operations

```
HashMap scores = new HashMap<>();
scores.put("Alice", 95);
scores.put("Alice", 99);           // Duplicate key → OVERWRITES!
scores.get("Alice");               // 99
scores.containsKey("Alice");       // true
for (Map.Entry e : scores.entrySet())
    System.out.println(e.getKey() + " -> " + e.getValue());
```

■ ■ 12 · JDBC (5/5 years — 10–20 marks) ★ HIGHEST PRIORITY ★

Step	Code	Purpose
1. Load Driver	<code>Class.forName("com.mysql.cj.jdbc.Driver");</code>	Register JDBC driver
2. Connect	<code>DriverManager.getConnection(URL, user, pass);</code>	Open DB connection
3. Create Stmt	<code>conn.createStatement();</code>	Prepare execution
4a. Query	<code>st.executeQuery("SELECT ...");</code> → <code>ResultSet</code>	Read data
4b. Update	<code>st.executeUpdate("INSERT/UPDATE/DELETE");</code> → <code>int</code> rows	Modify data
5. Close	<code>rs.close(); st.close(); conn.close();</code>	Release resources

```
static final String URL = "jdbc:mysql://localhost:3306/university";
static final String USER = "root";
static final String PASS = "password";

Connection conn = null;
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(URL, USER, PASS);
    // PreparedStatement – SAFER (prevents SQL injection)
    String sql = "INSERT INTO students VALUES (?, ?, ?)";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setInt(1, 101); ps.setString(2, "Alice"); ps.setFloat(3, 85.5f);
    ps.executeUpdate();

    // SELECT
    ResultSet rs = conn.createStatement().executeQuery("SELECT * FROM students");
    while (rs.next())
        System.out.println(rs.getInt("roll") + " " + rs.getString("name"));
} catch (ClassNotFoundException e) { System.out.println("Driver missing"); }
catch (SQLException e) { System.out.println(e.getMessage()); }
finally { if (conn!=null) conn.close(); }
```

Type	SQL Params	Precompiled?	SQL Injection Safe?
Statement	Hardcoded	✗ No	✗ No
PreparedStatement	? placeholders	✓ Yes	✓ Yes
CallableStatement	{call proc(?)}	✓ Yes	✓ Yes

■ ■ 13 · GUI SWING (4/5 years — 20 marks)

Component	Class	Purpose
Window	<code>JFrame</code>	Top-level container (main window)
Panel	<code>JPanel</code>	Group components
Button	<code>JButton</code>	Clickable button
Text Field	<code>JTextField</code>	Single-line input
Label	<code>JLabel</code>	Display text/image
Text Area	<code>JTextArea</code>	Multi-line input
Combo Box	<code>JComboBox</code>	Dropdown selection

```
// Minimal Swing Form Template
import javax.swing.*; import java.awt.*; import java.awt.event.*;

public class MovieForm extends JFrame implements ActionListener {
    JTextField tfTitle = new JTextField();
    JButton btnSubmit = new JButton("Submit");
    JTextArea taOut = new JTextArea();
```

```

MovieForm() {
    setTitle("Movie Form"); setSize(400,300);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new GridLayout(3,2,5,5));
    add(new JLabel("Title:")); add(tfTitle);
    add(btnSubmit);
    add(new JScrollPane(taOut));
    btnSubmit.addActionListener(this); // register listener
    setVisible(true);
}
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnSubmit)
        taOut.append(tfTitle.getText() + "\n");
}
public static void main(String[] args) { new MovieForm(); }
}

```

Layout	Behaviour
FlowLayout	Left-to-right, wraps to next line
BorderLayout	5 regions: N / S / E / W / Center
GridLayout	Equal-sized grid — ideal for forms
BoxLayout	Horizontal or vertical stack

14 · OUTPUT PREDICTION CHEATSHEET

Code Snippet	Output	Key Rule
int x=5,y; y=x++; sop(y+" "+x);	5 6	Post: use then increment
int x=5,y; y=++x; sop(y+" "+x);	6 6	Pre: increment then use
int a=5,b=2,c=3; sop(a+b*c-a/b);	9	b*c=6, a/b=2(int), 5+6-2=9
s1="Java"+"Lang"; s2.concat(s1); sop(s2);	AdvancedProgramming	concat result discarded
TreeSet: add Prog,Obj,Ori,Obj; iterate	Object Oriented Programming	Sorted + no duplicates
static x=0; 3 new A() each inc x; sop(x)	3	Static shared across objs
Thread t; t.start(); t.start();	IllegalThreadStateException	start() only once!

15 · MUST-MEMORISE SYNTAX PATTERNS

```

// 1. JDBC Connection (5/5 years)
Class.forName("com.mysql.cj.jdbc.Driver");
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/db","user","pass");

// 2. Thread via Runnable (4/5 years)
Thread t = new Thread(new MyRunnable());
t.start();

// 3. ArrayList (5/5 years)
ArrayList list = new ArrayList<>();
list.add(new MyClass()); list.get(0); list.set(0,obj); list.remove(0); list.size();

// 4. Exception skeleton
try { } catch (ExceptionType e) { } finally { }

// 5. Inheritance
class Child extends Parent { }
class MyClass implements Interface1, Interface2 { } // multiple interfaces OK

```

16 · COMMON MISTAKES — AVOID THESE

Wrong	Correct / Explanation
float f = 3.14	float f = 3.14f
str1 == str2	str1.equals(str2) (== compares refs, not content)
str.concat() alone	str = str.concat(...) (Strings are immutable)
TreeSet is insertion-ordered	TreeSet is SORTED (natural/alphabetical order)
thread.start() twice	IllegalThreadStateException — start() only ONCE

if (x = 5)	if (x == 5) (= is assignment, always true)
arr.length()	arr.length (arrays: no parentheses)
Forgot break in switch	Fall-through: ALL subsequent cases execute
Forgot close FileWriter	Data may not be flushed to disk!
Only change return type	NOT overloading if params same → compile error
abstract when interface needed	Use interface for multiple inheritance contract
throw vs throws mixed up	throw inside method body; throws in method signature

■ ■ 17 · EXAM STRATEGY

Topic	Freq	Max Marks	Priority	Study Hours
JDBC	5/5	20	★★★ CRITICAL	5–6
GUI Swing	4/5	20	★★★ CRITICAL	4
IO Streams	4/5	20	★★★ HIGH	3–4
Multithreading	5/5	10	★★★ HIGH	4–5
Collections	5/5	10	★★★ HIGH	4
Inheritance/Poly	5/5	10	★★ HIGH	4
Exception Handling	5/5	10	★★ HIGH	3
Classes/Encap	5/5	8	★★ HIGH	3
Packages/Interfaces	5/5	10	★★ HIGH	3
Strings/Arrays	5/5	4	★ MEDIUM	2–3
Data Types/Operators	5/5	4	★ MEDIUM	2

Day	Focus
Day 3 before exam	JDBC template + Swing form — write both from memory on paper
Day 2 before exam	Collections (TreeSet output, ArrayList CRUD) + all Section 14 predictions
Day 1 before exam	Exception/Thread syntax + OOP design + scan all Common Mistakes
In exam — code	Write imports first → class → main() → close all braces
In exam — output	Trace line-by-line on rough paper — never guess operator precedence